Use Case BRD

INVENTORY & SALES MANAGEMENT SYSTEM

1. Application Overview

Build an application for a small retail shop to maintain products and stock, record sales, generate receipts, and monitor low-stock items. Cashiers record sales while administrators manage the catalogue, staff accounts, suppliers, and reports. All transactions must be persisted in a database.

2. Objective

Demonstrate reliable inventory updates linked to sales, product and supplier management, basic reporting, and secure access for shop staff.

The build should demonstrate the following skills:

Skill area	What the application should demonstrate
Requirement understanding	Translate requirements into screens, APIs, data entities, and business rules.
Data modelling	Design tables/collections with keys, relationships, constraints, and appropriate data types.
Backend engineering	Implement APIs with validation, consistent error handling, and suitable status codes.
Frontend engineering	Build usable forms, lists, search/filter controls, and clear success/error states.
Data integrity	Enforce business rules on the server and database, including transaction-safe updates.
Security basics	Hash passwords, enforce server-side role permissions, and keep secrets out of source control.
Production readiness	Use configuration/environment variables, log errors, and document setup.
Deployment	Run locally or deploy to a suitable cloud/free-tier host with clear instructions.

3. Requirements and Data Flow

The application has three role groups. The functional requirements below define the expected minimum workflow; Must items should be completed first. Should and Could items are optional extensions.

3.1 User Roles

Role	How they get access	What they do
Cashier	Account created by Admin	Searches products, records sales, and views receipts for their transactions.
Inventory Staff	Account created by Admin	Updates stock receipts and monitors stock levels.
Admin/Manager	Seeded during setup	Manages products, suppliers, staff, adjustments, and sales reports.

3.2 Functional Requirements

ID	Role	Requirement	Priority
FR-01	All	Log in and log out and show a role-specific dashboard.	Must
FR-02	Cashier	Search active products by name, SKU, or category and view current price and stock.	Must
FR-03	Admin/Manager	Create, edit, and deactivate products with SKU, category, price, and reorder level.	Must

ID	Role	Requirement	Priority
FR-04	Inventory Staff	Record incoming stock with quantity, supplier, and received date.	Must
FR-05	System	Update stock accurately when incoming stock is recorded.	Must
FR-06	Cashier	Create a sale containing one or more products and quantities and generate a unique receipt number.	Must
FR-07	System	Reject sales that exceed available stock and prevent negative inventory.	Must
FR-08	Cashier	View and print or download a receipt showing items, quantities, unit prices, totals, and sale date.	Should
FR-09	Admin/Manager	View sales history and filter by date, cashier, or receipt number.	Must
FR-10	Inventory Staff	View low-stock products based on each product's reorder level.	Must
FR-11	Admin/Manager	Create and update supplier records and view stock receipts by supplier.	Should
FR-12	Admin/Manager	Record a stock adjustment with reason, quantity change, date, and staff member.	Must
FR-13	Admin/Manager	View summary totals for sales, items sold, and low-stock products over a selected period.	Could
FR-14	System	Maintain a transaction or audit record for sales and stock adjustments.	Should

3.3 Business Rules

• Each SKU must be unique, and product price and quantity must be validated on the backend.

• A sale and its stock deductions must be committed in the same database transaction.

• Stock adjustments must include a reason and must not result in negative stock unless an explicitly approved correction workflow is implemented.

• Cashiers can access their own sales; managers can view all sales and inventory records.

• Deactivated products cannot be included in new sales, but historic receipts remain accessible.

3.4 Non-Functional Requirements

Area	Requirement
Security	Passwords must be hashed; protected APIs must enforce role permissions on the server.
Validation	Validate required fields, formats, dates, quantities, and business constraints on frontend and backend.
Error handling	Display understandable messages; use appropriate HTTP status codes such as 400, 401, 403, 404, and 409.
Logging	Log server errors and key operations with timestamps; do not expose stack traces to users.
Configuration	Read database connection strings, ports, and secrets from environment variables or configuration.
Persistence	Store application data in a database; records must survive application restart.
Usability	Provide clear navigation, loading states, empty states, and confirmation before destructive actions.

4. Setup and Deliverables

4.1 Setup Requirements

README, .env.example, schema/migrations, seed script with an admin, cashier, inventory staff, suppliers, and sample products.

#	Deliverable
1	Git repository containing source code and a README with setup and run instructions.
2	Database schema, migrations, seed script, and sample login details.

#	Deliverable
3	An .env.example file with variable names only; do not commit real secrets.
4	API documentation or a table listing endpoints, methods, access roles, and example responses.
5	A short design note describing key decisions, assumptions, known gaps, and possible improvements.

4.2 Ground Rules

• Any suitable programming language, framework, and database may be used; low-code-only implementations are not the goal.

• AI coding tools may be used, but the developer should understand and be able to explain the code they submit.

• Use Git with regular, meaningful commits throughout development.

• Record unclear requirements and document reasonable assumptions instead of silently ignoring them.

• Keep the scope manageable: implement the Must requirements before attempting optional enhancements.

Document end